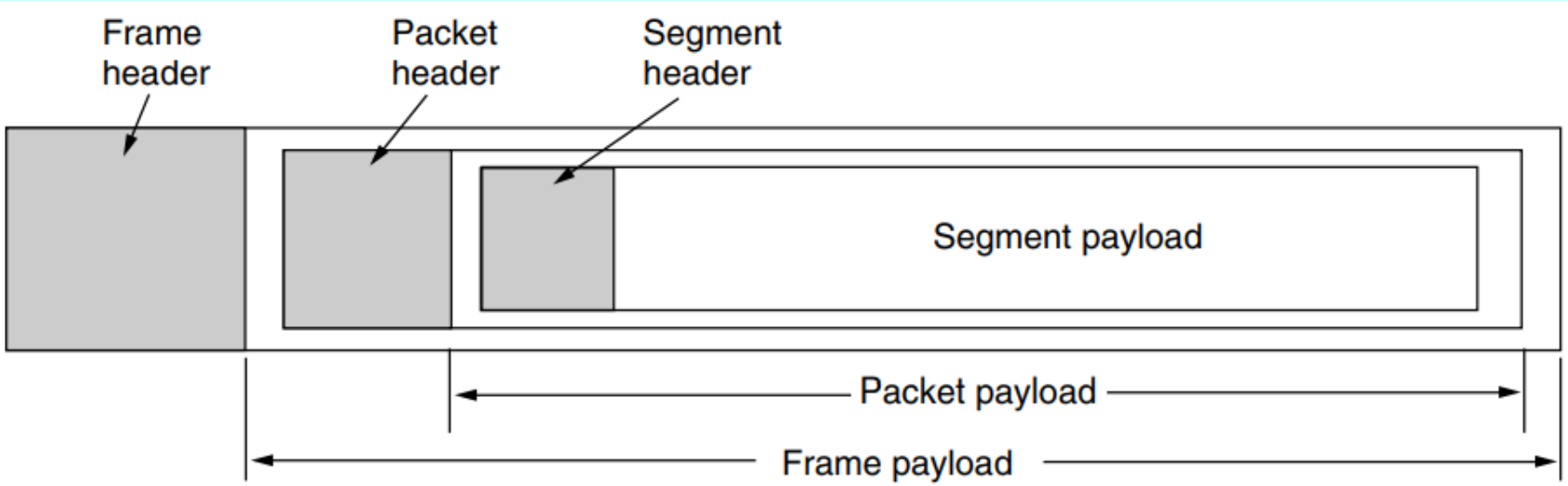
Transport Layer

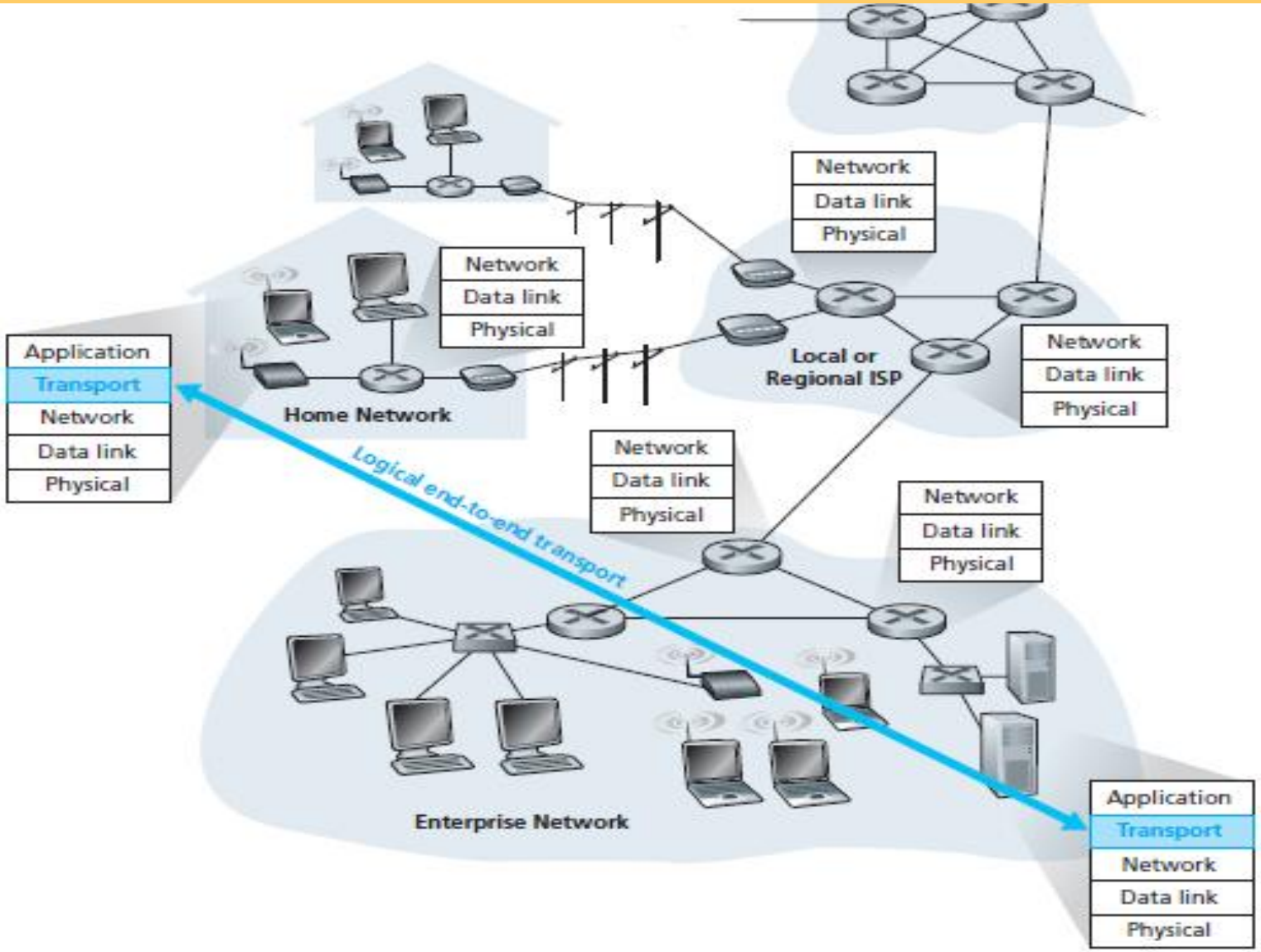
✓A transport-layer protocol provides for logical communication between applications processes of source machine and destination machine.

✓The hardware and/or software within the transport layer that does the work is called the **transport entity**. **(**transport layer entity দুই end মেশিনে মুখপাত্র হিসেবে কাজ করে**)**

✓We will use the term **segment** for messages sent from transport entity to transport entity.



The transport layer provides logical rather (lot of buffer or memory and wide range of link types between them) than physical communication (short circuit no buffer) between application processes.



Household Analogy

✓Consider two houses, one on the Dhaka and the other on the Khulna, with each house being home to a **dozen kids**. The kids in the Dhaka household are cousins of the kids in the Khulna household.

✓ The kids in the two households love to write to each other—each kid writes each cousin every week, with each letter delivered by the traditional postal service in a separate envelope. Thus, each household sends **144 letters** (12×12 i.e. all to all relation) to the other household every week.

✓In each of the households there is one caretaker—Roshid in the Khulna house and Hasan in the Dhaka house—responsible for mail collection and mail distribution.

✓Each week Roshid visits all brothers and sisters, collects the mail, and gives the mail to a postal-service mail carrier, who makes daily visits to the house.

✓When letters arrive at the Khulna house, Roshid also has the job of distributing the mail to her brothers and sisters. Hasan has a similar job on the Dhaka.

✓In this example, the postal service provides logical communication between the two houses (using intermediate post offices) —the postal service moves mail from house to house, not from person to person.

✓On the other hand, Roshid and Hasan provide logical communication among the cousins. Roshid and Hasan pick up postal mail from, and deliver mail to, their brothers and sisters.

- ✓ **Application messages = all letters inside the envelopes**
- ✓ **Transport layer's messages = each letter inside an envelope**
- ✓ **Transport layer's header = envelopes**
- ✓ **Transport-layer protocol = Roshid** and **Hasan**
- ✓ **Processes = cousins (who order Hansan/Roshid to send the letter or message)**
- ✓ **Hosts (also called end systems) = houses**
- ✓ **Network-layer protocol = postal service (including mail carriers), who deals with several post offices hop-by-hop until reach the destination house.**
- ✓ **IP address = Address of the houses**

✓On the sending side, the transport layer converts the application-layer messages it receives from a **sending application process** into **transport-layer packets**, known as **transport-layer segments** in Internet terminology.

✓This is done by (possibly) breaking the application messages into smaller chunks and adding a transport-layer header to each chunk to create the transport-layer segment.

✓The Internet has two main protocols in the transport layer, a **connectionless** protocol and a **connection-oriented** one. In the following sections we will study both of them. The connectionless protocol is **UDP (User Datagram Protocol)**. The connection-oriented protocol is **TCP (Transmission Control Protocol)**.

Basic Functions of Transport layer:

- ✓ Connection establishment
- ✓ Disconnect two end system
- ✓ Multiplexing de-multiplexing of message at end machines
- ✓ Congestion control

Connection Establishment

- ✓ Establishing a connection sounds easy, but it is actually surprisingly tricky. At first glance, it would seem sufficient for one transport entity to just send a CONNECTION REQUEST TPDU to the destination and wait for a CONNECTION ACCEPTED reply.

- ✓ The problem occurs when the network can lose, store, and duplicate packets. This behavior causes serious complications.

- The worst possible nightmare is as follows. A user establishes a connection with a bank (online money transfer), sends messages telling the bank to transfer a large amount of money to the account of a not-entirely-trustworthy person, and then releases the connection.

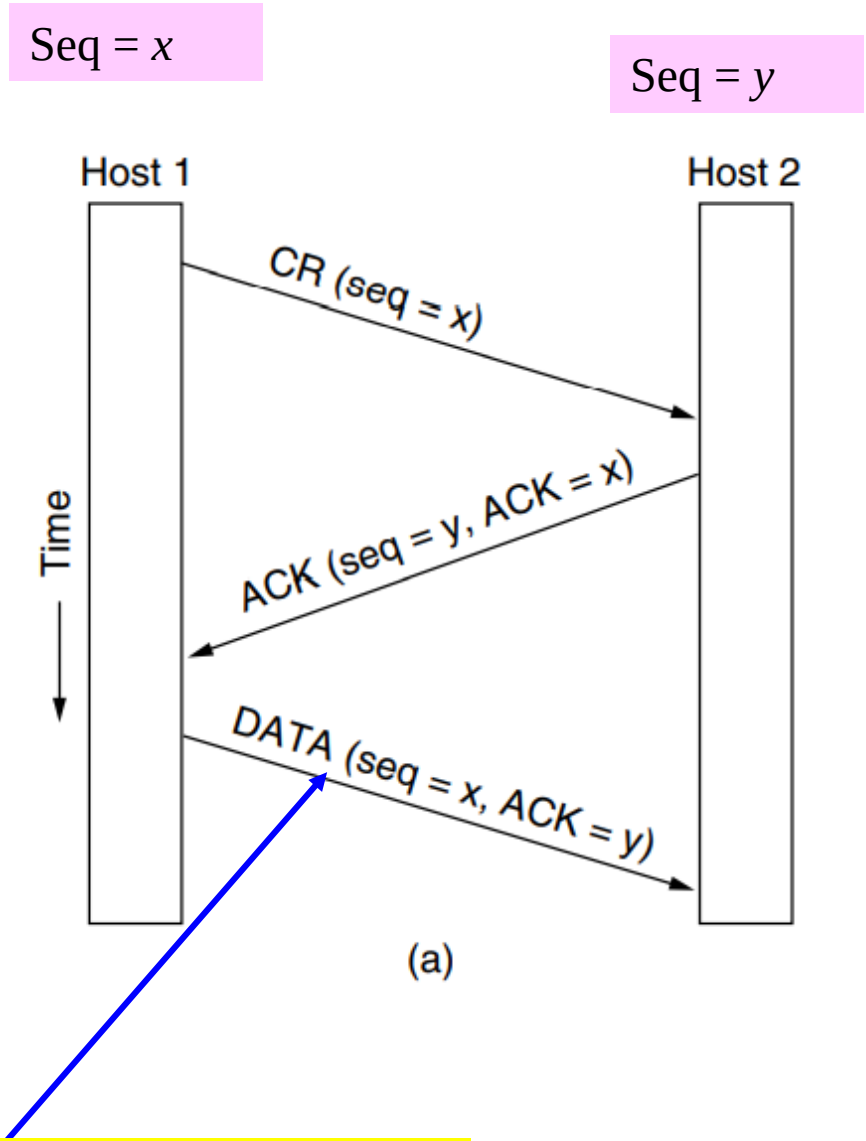
- Unfortunately, each packet in the scenario is duplicated and stored in the subnet. After the connection has been released, all the packets pop out of the subnet and arrive at the destination in order, asking the bank to establish a new connection, transfer money (again), and release the connection.

- The bank has no way of telling that these are duplicates. It must assume that this is a second, independent transaction, and transfers the money again.

✓Tomlinson (1975) introduced the **three-way handshake.** The normal setup procedure when host 1 initiates is shown in fig. below. Host 1 chooses a sequence number, *x*, and sends a **CONNECTION REQUEST TPDU** containing it to host 2.

✓Host 2 replies with an **ACK TPDU** acknowledging *x* and announcing its own initial sequence number, *y*.

✓Finally, host 1 acknowledges host 2's choice of an initial sequence number in the first **DATA TPDU** that it sends.

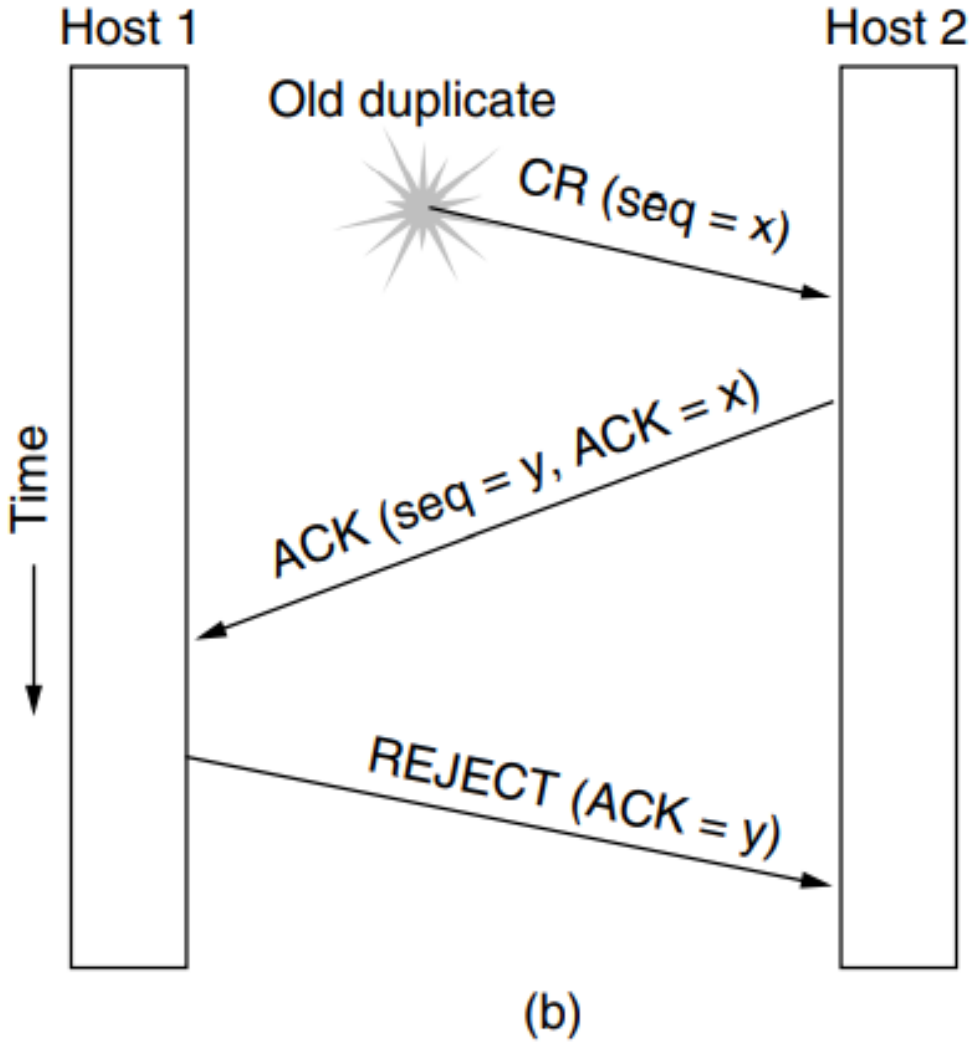


seq = *x* and ACK = *y* is sent only on first data segment but not on the subsequent segments.

✓Now let us see how the three-way handshake works in the presence of **delayed duplicate control TPDUs**.

✓In Fig. below, the first TPDU is a delayed duplicate CONNECTION REQUEST from an old connection. This TPDU arrives at host 2 without host 1's knowledge. Host 2 reacts to this TPDU by sending host 1 an ACK TPDU, in effect asking for verification that host 1 was indeed trying to set up a new connection.

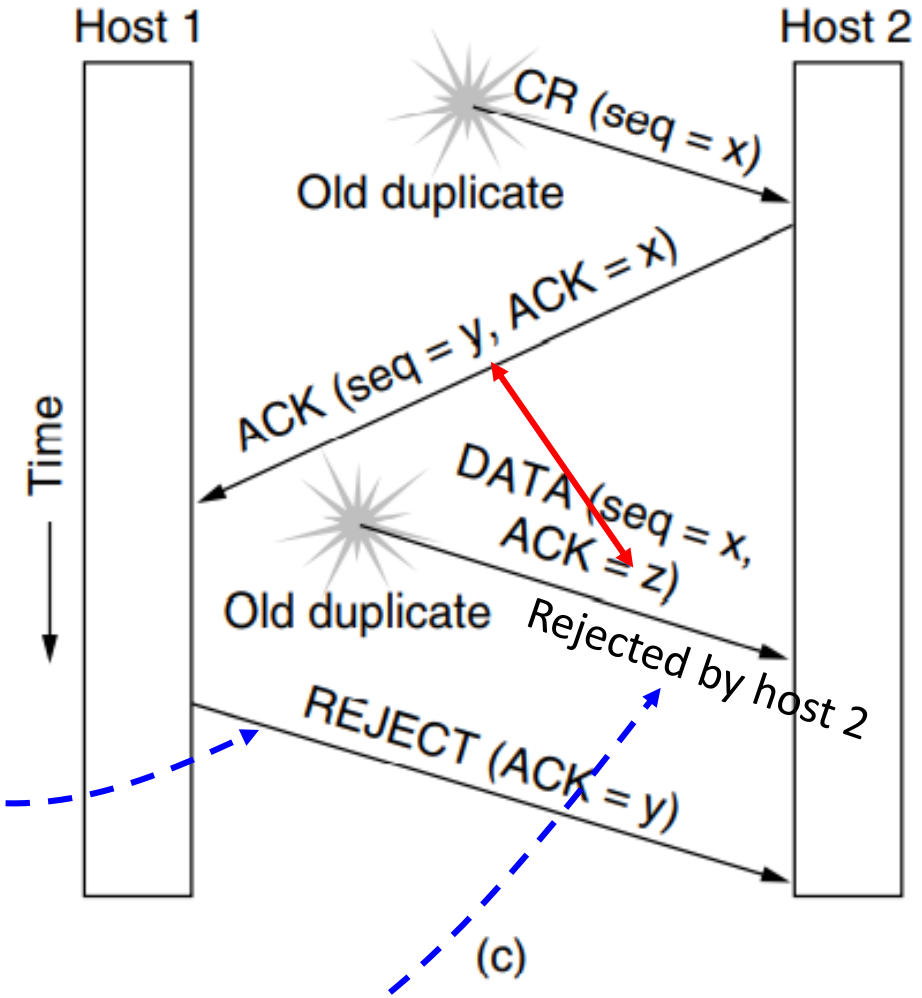
✓When host 1 rejects host 2's attempt to establish a connection, host 2 realizes that it was tricked by a delayed duplicate and abandons the connection. In this way, a delayed duplicate does no damage.



The worst case is when **both a delayed CONNECTION REQUEST (CR)** and **an ACK are floating around in the subnet**. This case is shown in Fig. below.

✓As in the previous example, host 2 gets a delayed **CONNECTION REQUEST (CR)** and replies to it using *y* as the initial sequence number. This CR will be rejected by Host-1.

✓When the second delayed TPDU:
DATA(seq = x, ACK = z) arrives at host 2, the fact that z has been acknowledged rather than y tells host 2 that this, is an old duplicate.



Connection Release

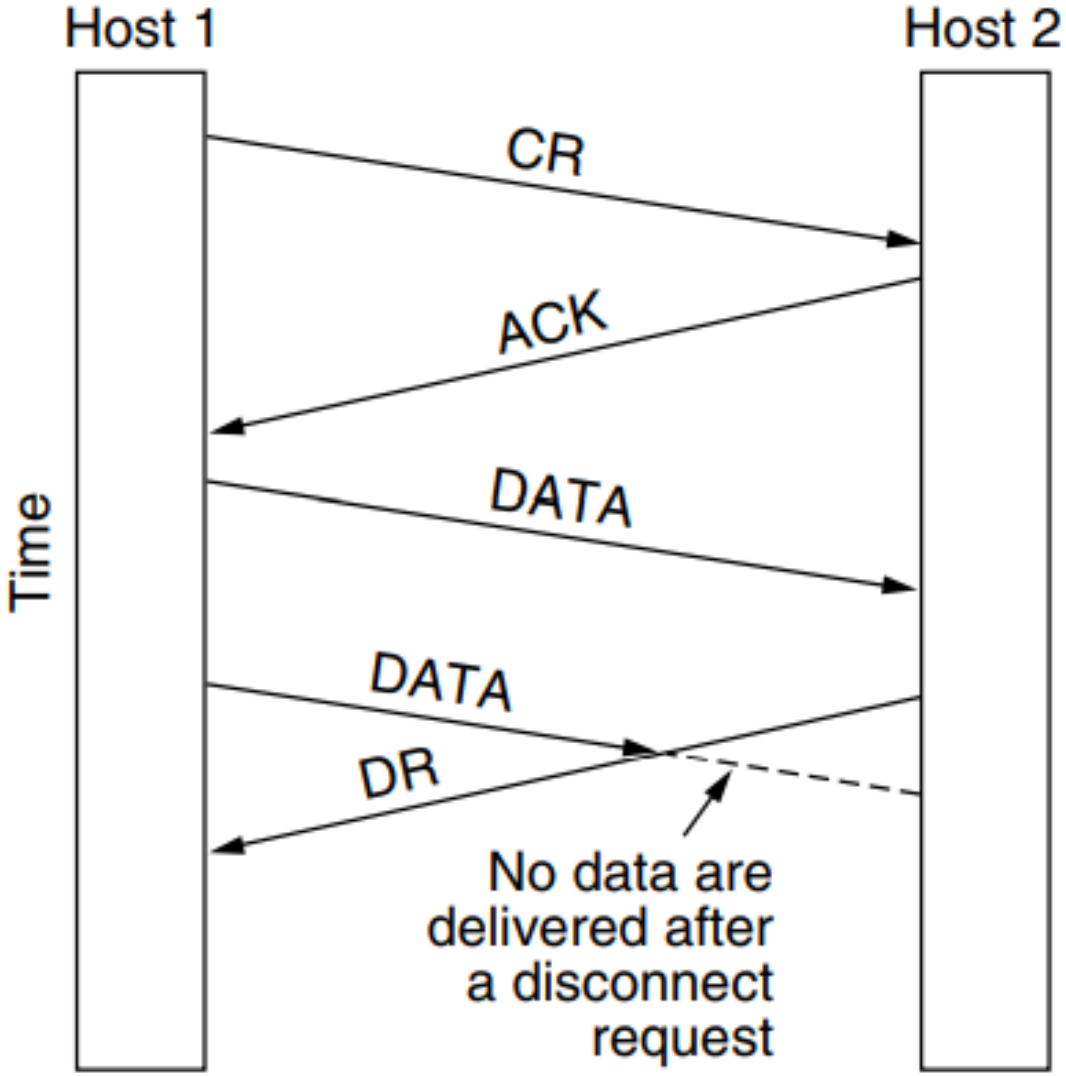
✓Releasing a connection is easier than establishing one. Nevertheless, there are more pitfalls than one might expect. There are two styles of terminating a connection: asymmetric release and symmetric release.

✓**Asymmetric release** is the way the telephone system works: when one party hangs up (end the phone call), the connection is broken. **Symmetric release** treats the connection as two separate unidirectional connections and requires each one to be released separately.

- ✓ Asymmetric release is abrupt and may result in data loss. Consider the scenario of Fig. below. After the connection is established, host 1 sends a TPDU that arrives properly at host 2. Then host 1 sends another TPDU. Unfortunately, host 2 issues a DISCONNECT before the second TPDU arrives. The result is that the connection is released and data are lost.

- ✓ Clearly, a more sophisticated release protocol is needed to avoid data loss.

CR → Connection Request



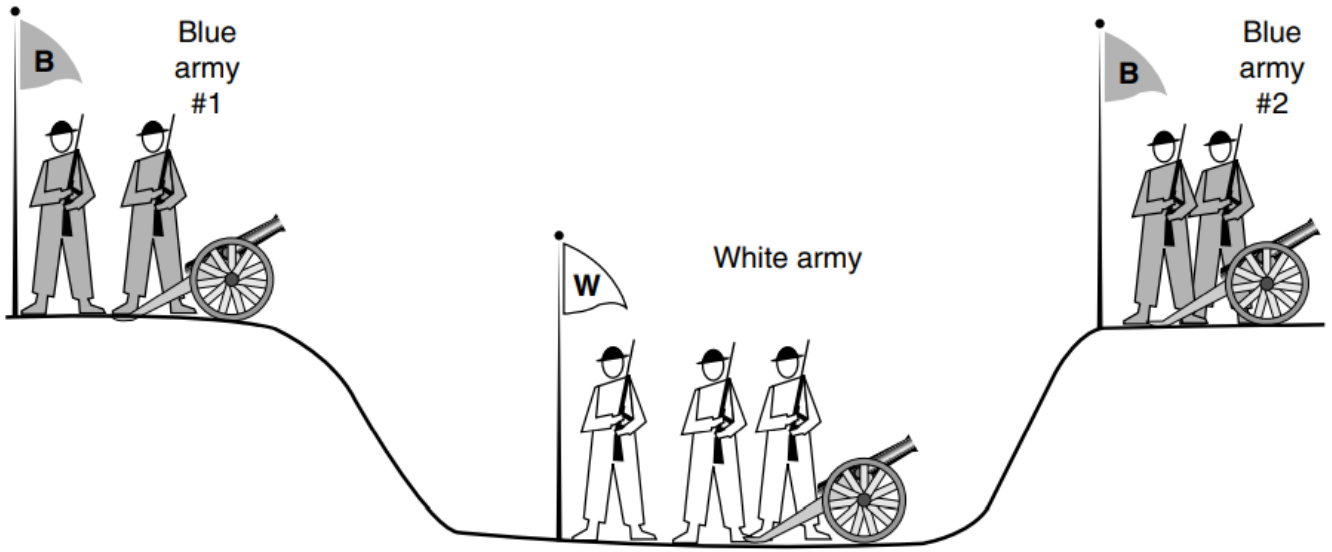
DR → Disconnection Request

✓One way is to use **symmetric** release, in which each direction is released independently of the other one. Here, a host can continue to receive data even after it has sent a DISCONNECT TPDU.

✓Symmetric release does the job when each process has a fixed amount of data to send and clearly knows when it has sent it.

একপাশের Host তার ডাটা পাঠানো শেষ হওয়ার পর সে নিজেকে Disconnect করে বের হয়ে গেল; কিন্তু অন্য পাশের Host যদি আশা করে সে এখনো ডাটা পাঠাবে এবং রিসিভ করবে তাহলে প্রচুর প্যাকেট লস হবে । উভয় হোস্ট যদি আগের থেকেই জানে আমি কতটুকু ডাটা পাঠাবো এবং কতটুকু রিসিভ করব তাহলে একজন ডাটা পাঠানো শেষ করার পর সে Disconnected হয়ে গেল কিন্তু ডাটা রিসিভ করেই যাচ্ছে কারণ সে জানে ওইপাশ থেকে কতটুক ডাটা আসবে । সর্বশেষে ওই ডাটা আসার পর সে ডাটা রিসিভ করো বন্ধ করে দিতে পারে । পুরো বিষয়টি অন্যপাশের হোস্টের জন্যেও প্রযোজ্য ।

✓One can envision a protocol in which host 1 says: I am done. Are you done too? If host 2 responds: I am done too. Goodbye, the connection can be safely released. Unfortunately, this protocol does not always work. There is a famous problem that illustrates this issue. It is called the **two-army problem.**



Two-way handshaking experiences **two-army problem.**

Suppose that the commander of blue army #1 sends a message reading: "I propose we attack at dawn on March 29. How about it?" Now suppose that the message arrives, the commander of blue army #2 agrees, and his reply gets safely back to blue army #1. Will the attack happen? Probably not, because commander #2 does not know if his reply got through. If it did not, blue army #1 will not attack, so it would be foolish for him to charge into battle

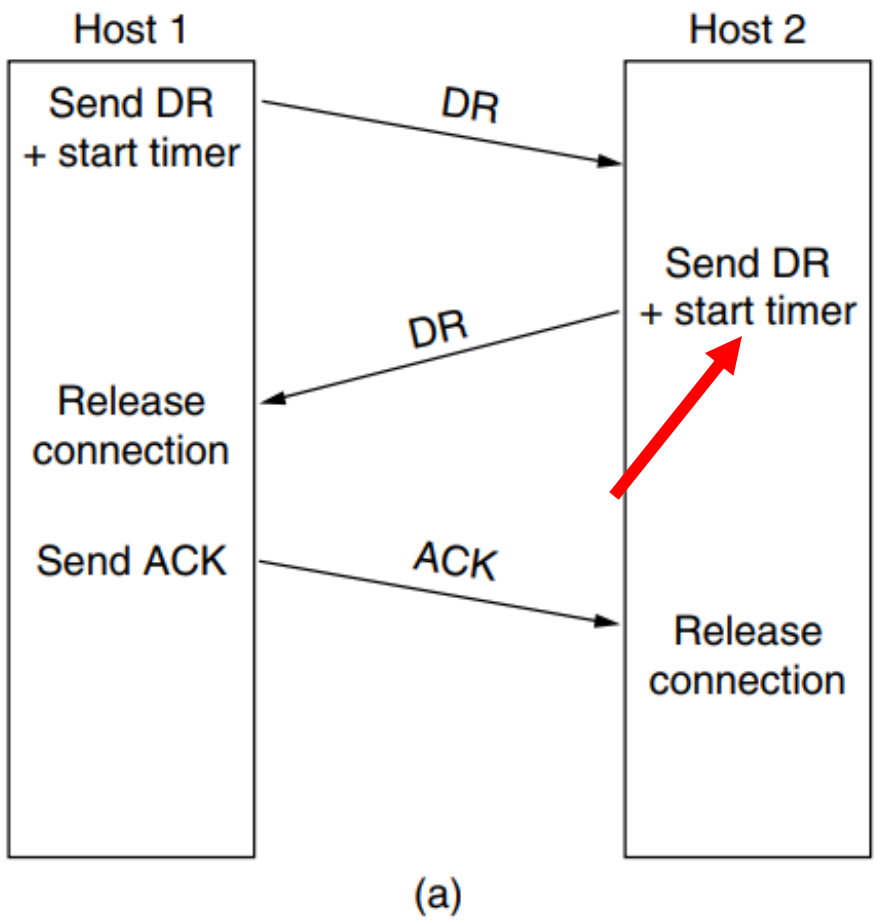
✓We will next consider using a three-way handshake for four scenarios of releasing connection.

✓In Fig. (a), we see the normal case in which one of the users sends a DR (**DISCONNECTION REQUEST**) TPDU to initiate the connection release.

✓When DR arrives, the recipient sends back a DR TPDU, too, and **starts a timer**, just in case its DR is lost.

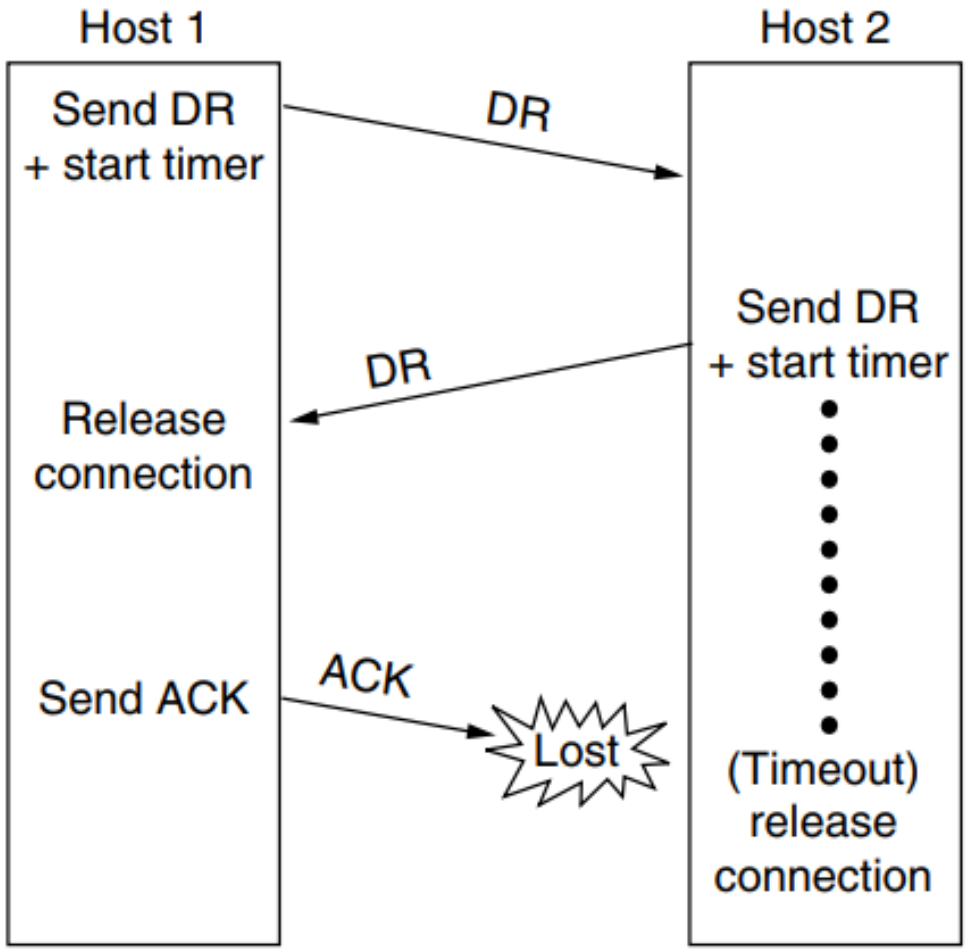
✓When this DR arrives, the original sender sends back an ACK TPDU and releases the connection.

✓Finally, when the ACK TPDU arrives, the receiver also releases the connection.



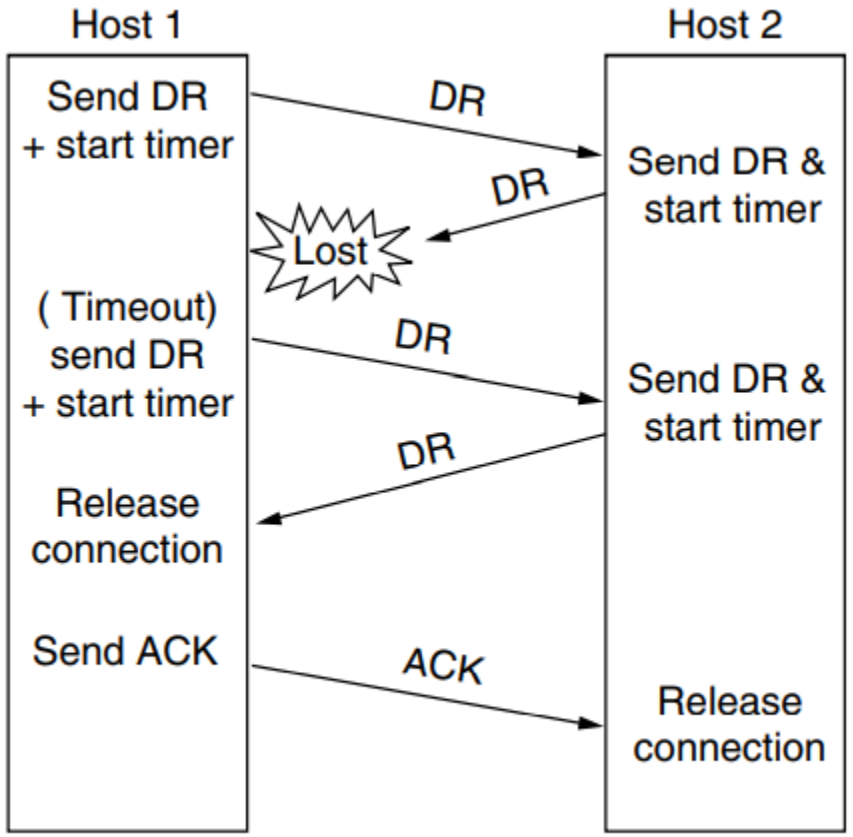
(a)

If the final ACK TPDU is lost, as shown in Fig. 6-14(b), the situation is saved by the timer. When the timer expires, the connection is released anyway.



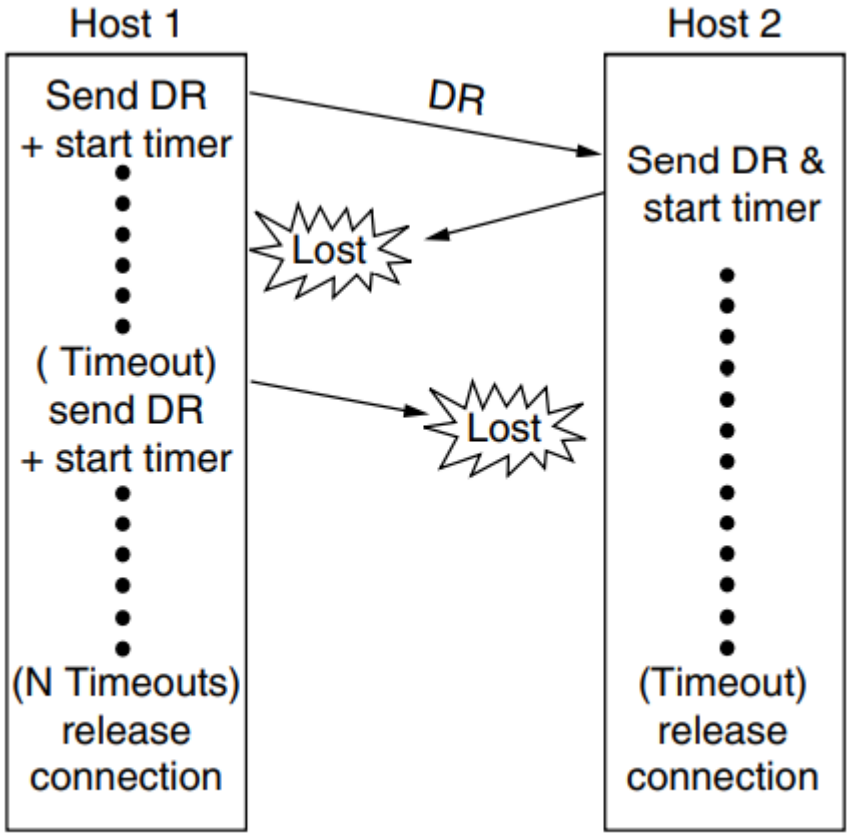
(b)

Now consider the case of the second DR being lost. The user initiating the disconnection will not receive the expected response, will time out, and will start all over again. In Fig. (c) we see how this works, assuming that the second time no TPDUs are lost and all TPDUs are delivered correctly and on time.



(c)

Our last scenario, Fig. (d), is the same as Fig. (c) except that now we assume all the repeated attempts to retransmit the DR also fail due to lost TPDUs. After N retries, the sender just gives up and releases the connection. Meanwhile, the receiver times out and also exits.

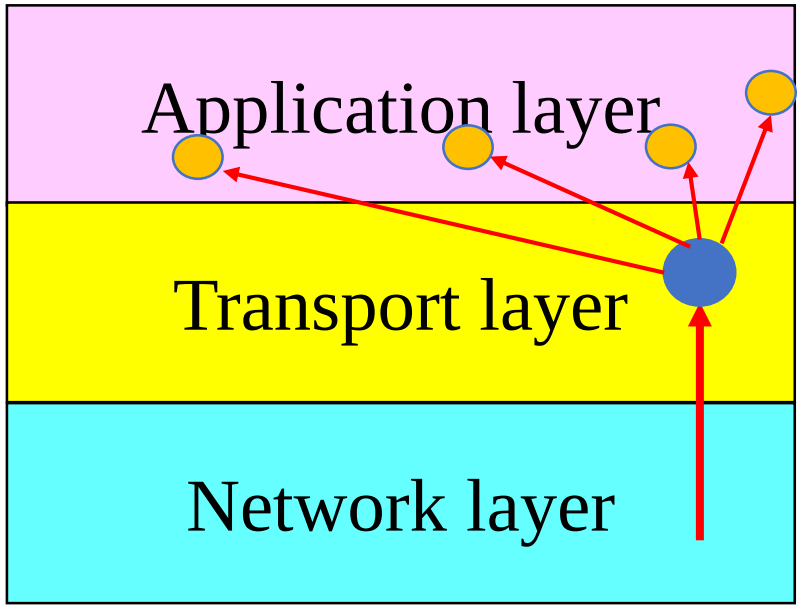


(d)

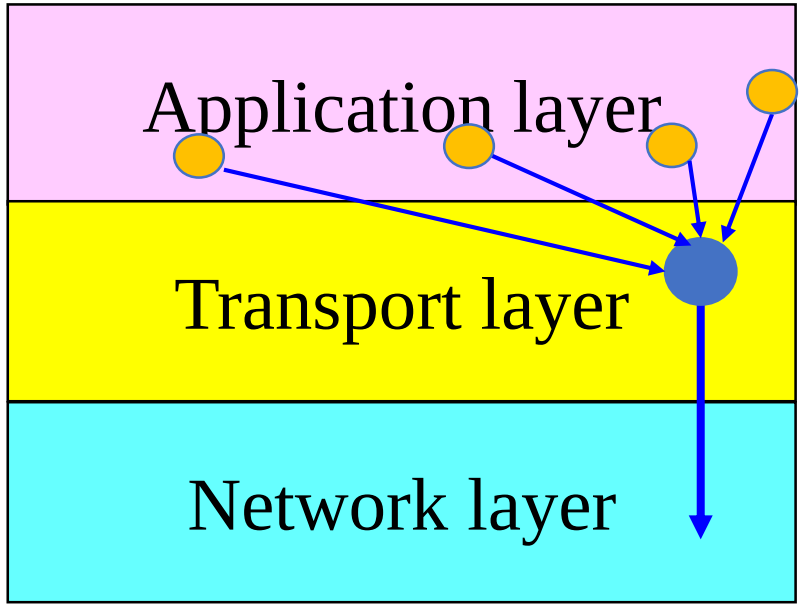
Multiplexing and Demultiplexing

- ✓ When the transport layer in a computer receives data packet from the network layer below of it destined to N corresponding processes. Above phenomenon is called **demultiplexing** (S/P converter) at transport layer.

- ✓ Similarly the job of combining data segments from different processes and deliver them to network layer is called **multiplexing** (P/S converter) at transport layer.



Demultiplexing

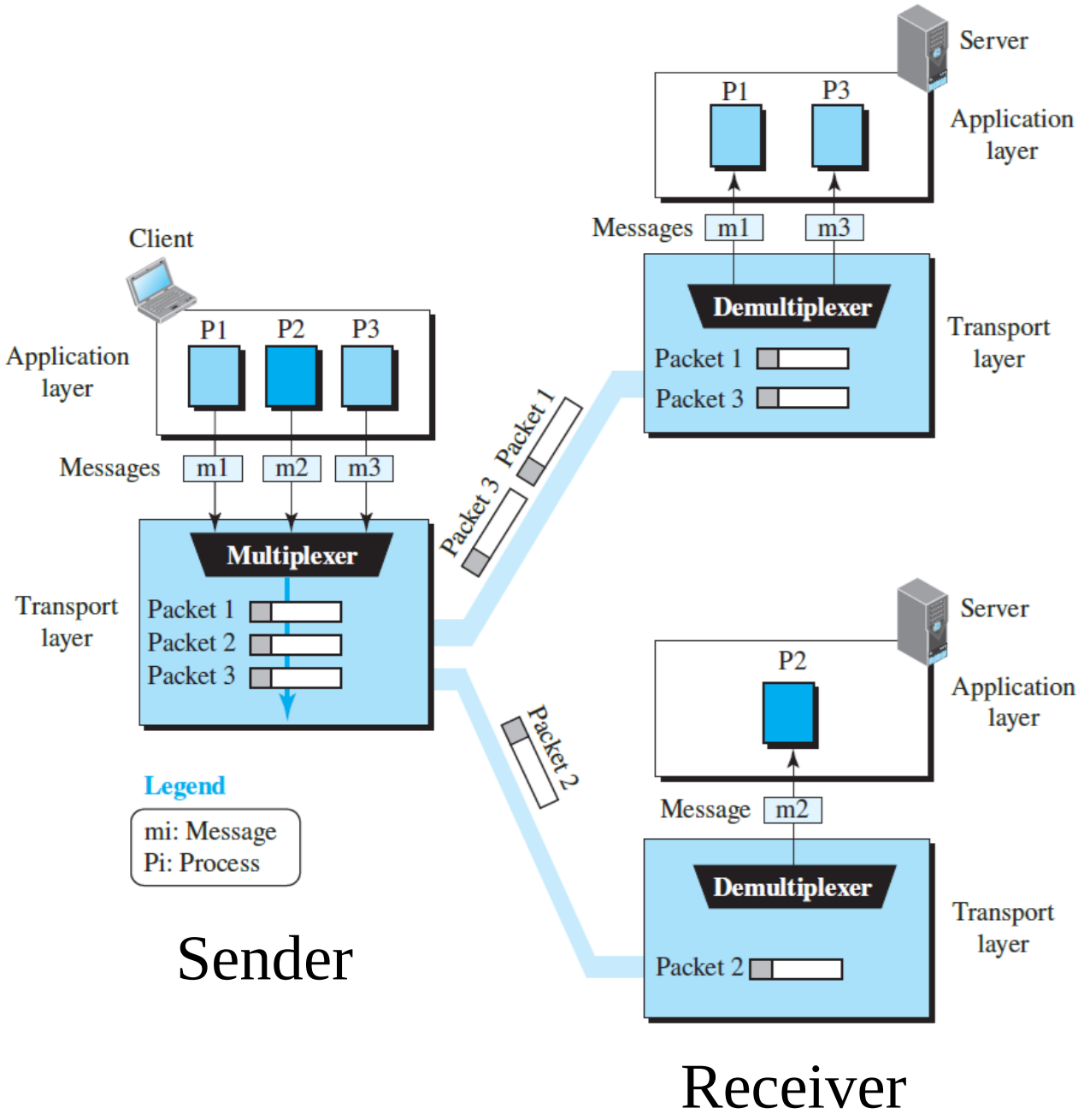


Multiplexing

- ✓ The local host and the remote host are defined using IP addresses.
- ✓ To define the processes, we need second identifiers, called **port numbers**. In the TCP/IP protocol suite, the port numbers are integers between 0 and 65,535 (16 bits).
- ✓ For a client the port number (process id at client side) "short-lived" and is used because the life of a client is normally short.
- ✓ TCP/IP has decided to use universal port numbers for servers relevant to each application (e-mail, DNS, client-server etc.).

HTTP(which uses port number 80) and
FTP(which uses port number 21)

Three client processes are running at the client site: P1, P2, and P3. The processes P1 and P3 need to send requests to the corresponding server process running in a server. The client process P2 needs to send a request to the corresponding server process running at another server.



Congestion Control

Regulating the Sending Rate

✓When the load offered (offered traffic) to any network is more than it can handle (carried traffic), congestion builds up.

✓When a connection is established, a suitable **window** size (amount of data can be sent without acknowledgement, initially the number of TCP packets) has to be chosen. The receiver can specify a **window** based on its buffer size.

✓If the sender sticks to the receiver's window size, problems will not occur due to buffer overflow at the receiving end, but they may still occur due to internal congestion within the network.

- ✓ In Fig.(a)-(b), we see this problem illustrated hydraulically. In Fig. (a), we see a thick pipe leading to a small-capacity receiver. As long as the sender does not send more water than the bucket can contain, no water will be lost.

- ✓ In Fig. (b) the limiting factor is not the bucket capacity, but the internal carrying capacity of the network. If too much water comes in too fast, it will back up and some will be lost (in this case by overflowing the funnel).

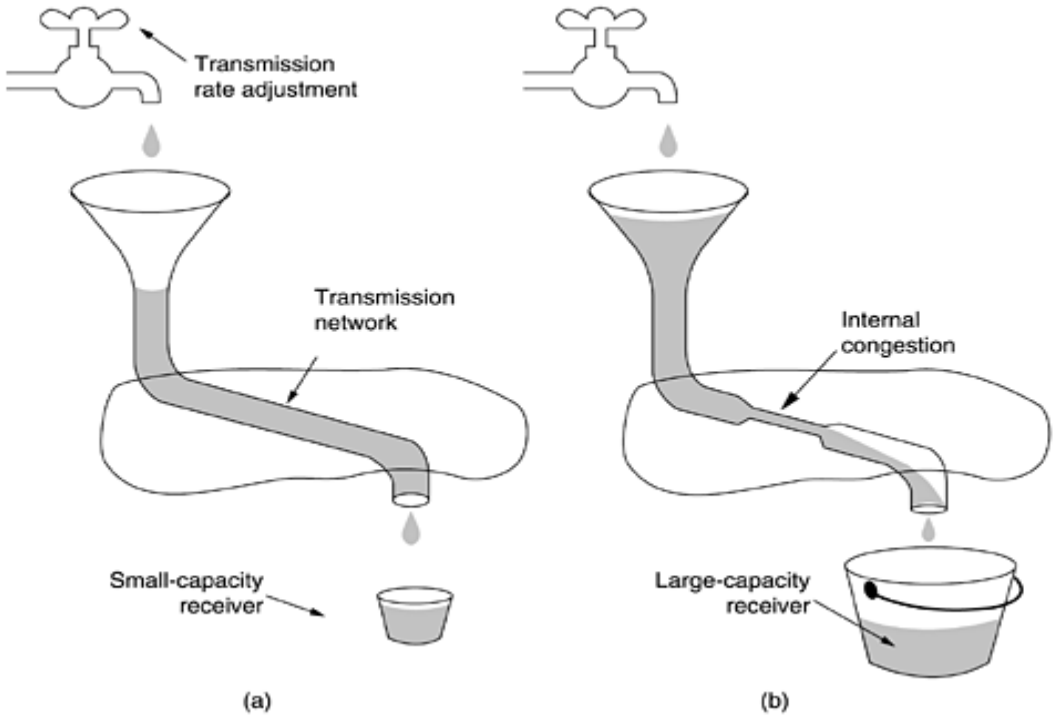


Figure (a) A fast network feeding a low-capacity receiver. (b) A slow network feeding a high-capacity receiver.

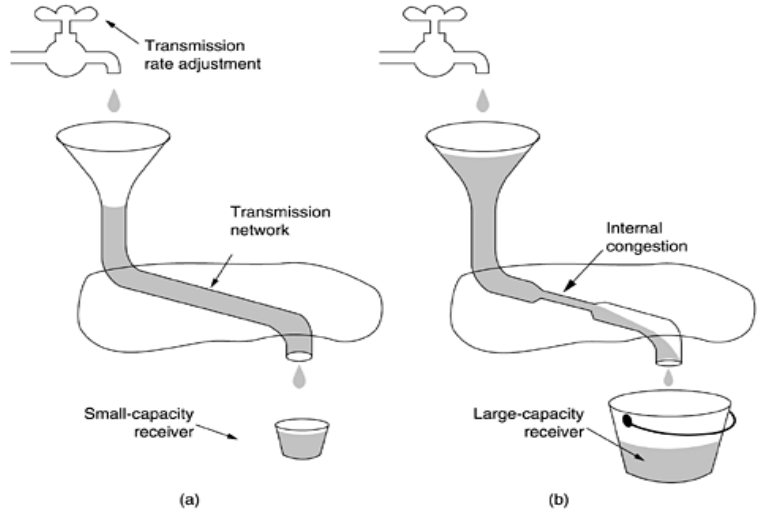
✓The Internet solution is, to realize that two potential problems : network capacity and receiver capacity and to deal with each of them separately. To do so, each sender maintains two windows: **the window the receiver has granted** and **the congestion window**.

- ✓ The number of segments to transmit, TCP uses a variable called a *congestion window*, ***cwnd***, whose size is controlled by the congestion situation in the network.

- ✓ Another variable or window *rwnd* (receive window) related to the limited buffer at the end (related to buffer flow like sliding window of LLC).

- ✓ The actual size of the window is the minimum of these two.

  Actual window size = minimum (*rwnd*, *cwnd*)



TCP Congestion Control

To reached at the optimum size of cwnd two widely used algorithms (congestion control algorithms) are:

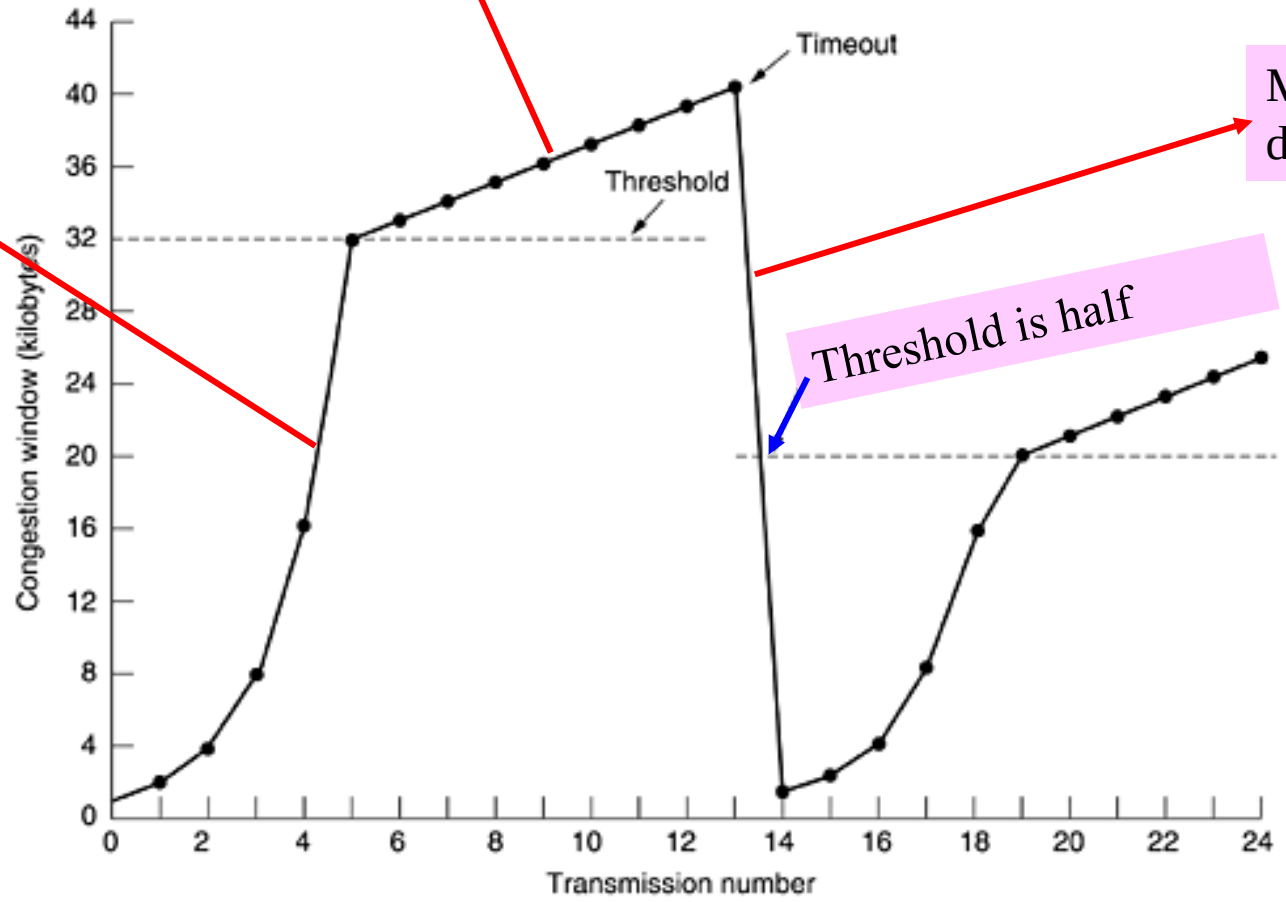
- ✓ TCP Tahoe
- ✓ TCP Reno, will be shown graphically.

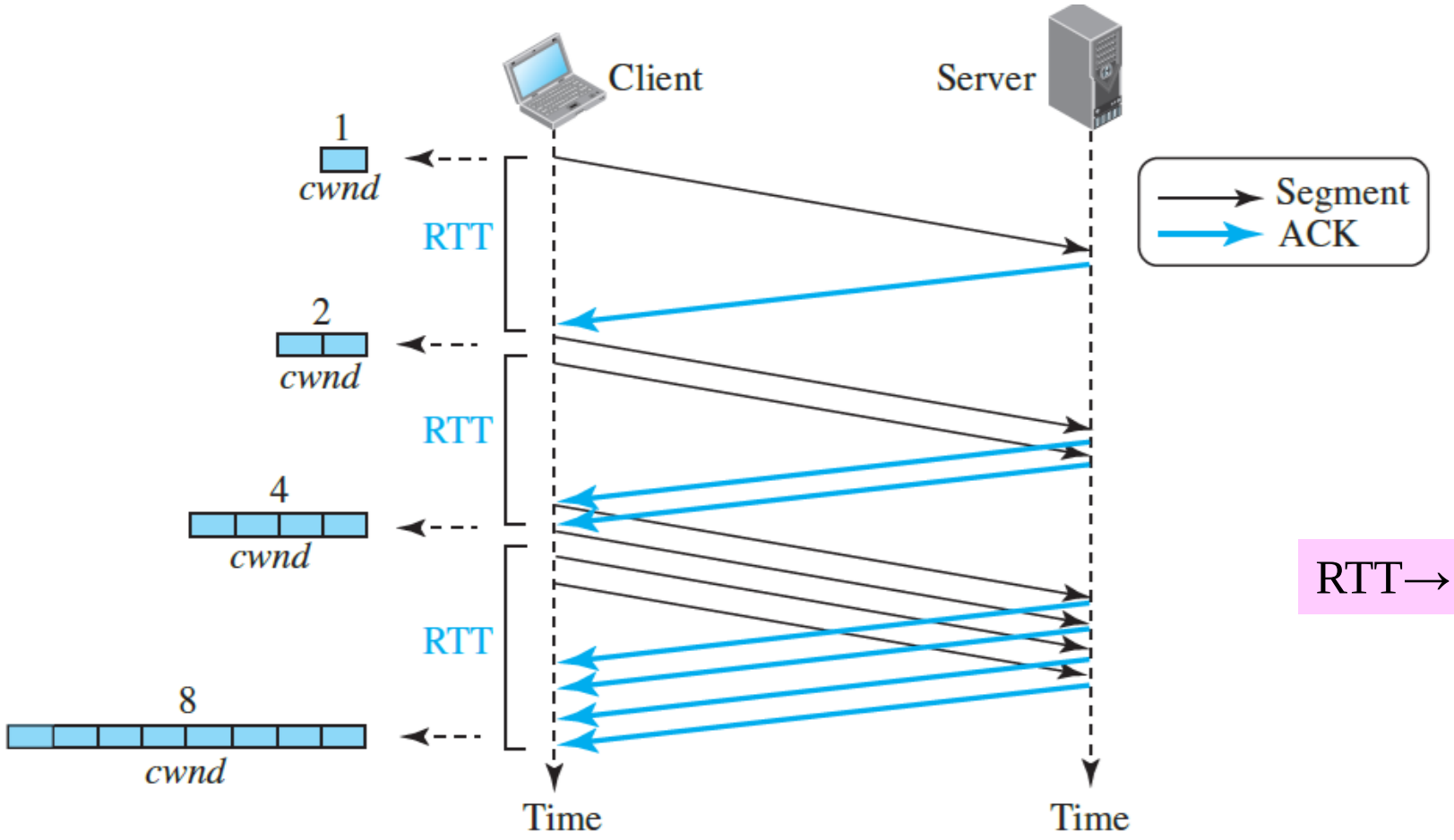
Internet congestion control algorithm of TCP Tahoe

Slow start/ exponential increase.

Additive increase or linear increase or congestion avoidance phase.

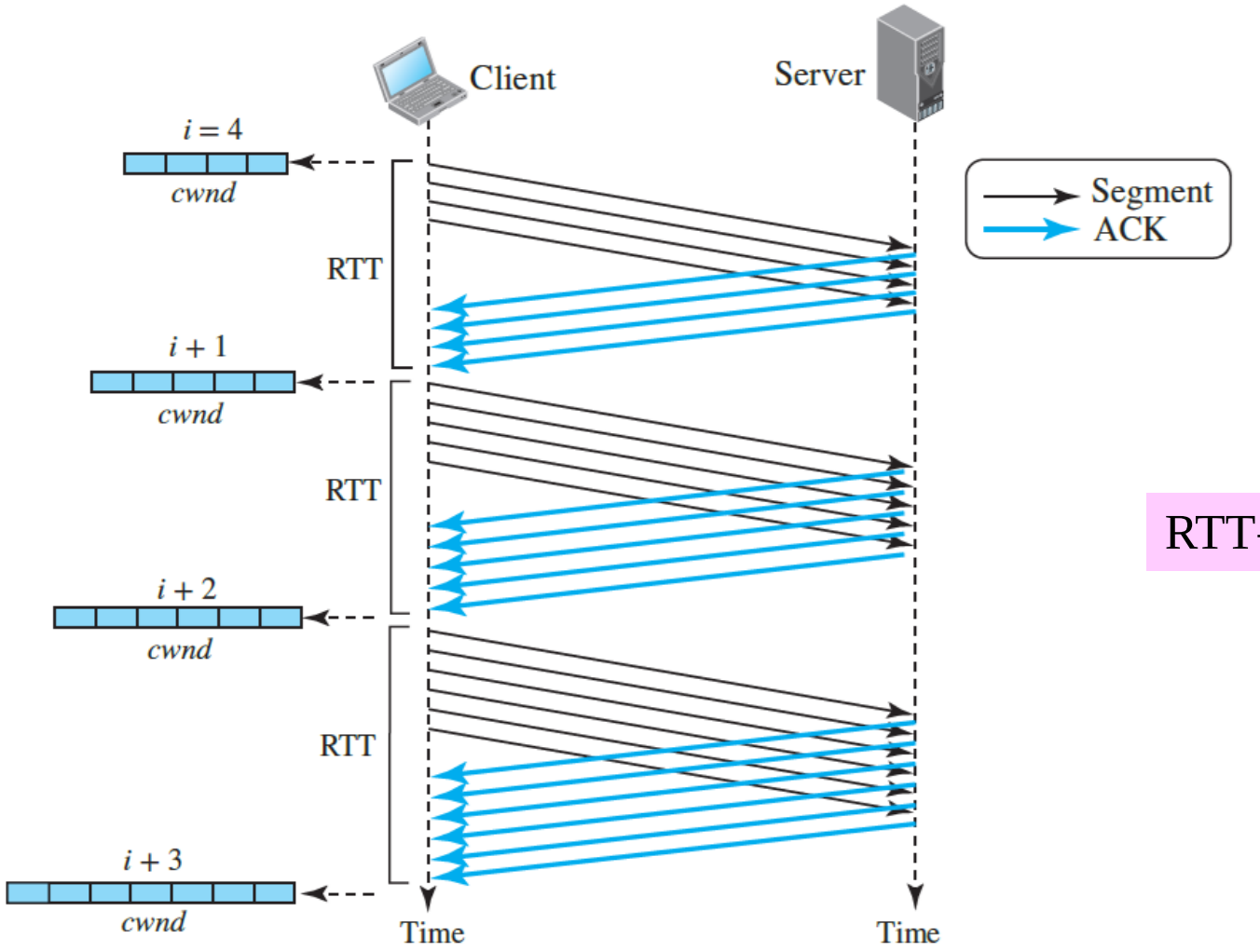
Multiplicative decrease.





RTT→ Round Trip Time

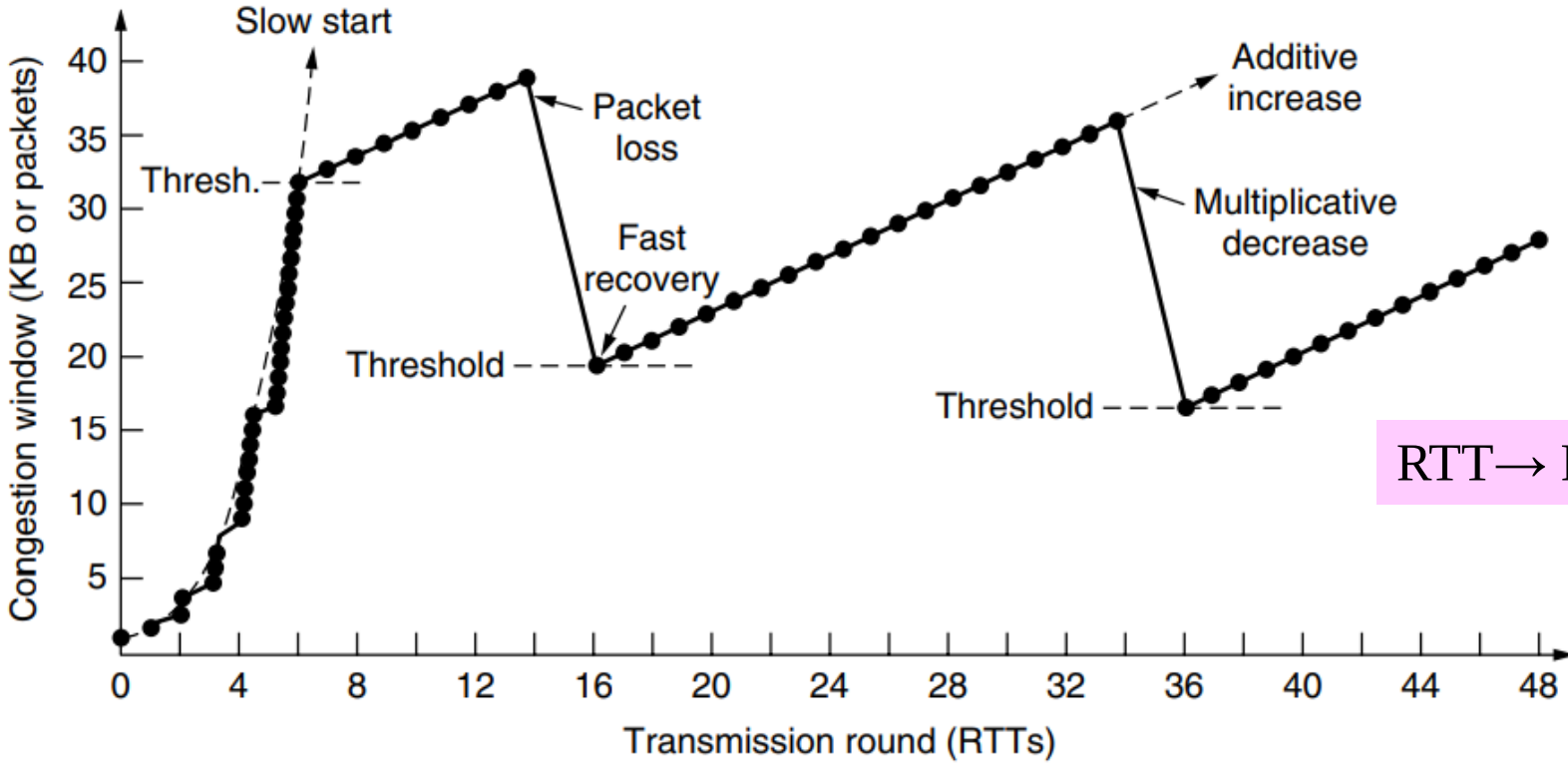
Slow start from an initial congestion window of one segment



RTT→ Round Trip Time

Additive increase from an initial congestion window of one segment

In **TCP Reno**, instead of repeated slow starts, the congestion window of a running connection follows a sawtooth pattern of additive increase (by one segment every RTT) and multiplicative decrease (by half in one RTT). This type of TCP congestion control is often referred to as an additive-increase, multiplicative decrease (AIMD) form of congestion control.



RTT→ Round Trip Time

Fast recovery and the sawtooth pattern of TCP Reno